Concurrent Neural Signal Processing for EEG Spindle Detection

Apurv Sanjay Deshpande
Department of Computer Engineering
Virginia Tech
apurvsd@vt.edu

Hari Makarand Sumant
Department of Computer Engineering
Virginia Tech
harisumant@vt.edu

***Abstract*—There have been plenty of studies for the detection of sleep spindles in EEG signals using machine learning techniques. Such methods use artificial neural networks (ANNs) and have demonstrated promising results. However, such methods may take a long time for training the model. There has not been much exploration to leverage parallel programming. This scenario is particularly useful for large-scale or real-time data analysis. In our project, we propose an approach that makes use of parallel processing techniques for training the neural network. We aim to achieve speedups for the training process, while maintaining the accuracy of the baseline model. By implementing a lock-based parallelization strategy in Python, we demonstrate the feasibility and effectiveness of accelerating ANN training without compromising model performance. This work bridges the gap between traditional machine learning-based spindle detection while using benefits of parallel computing.**

***Index Terms*—ANN, Neural Networks, EEG, Spindle Detection, Parallel processing, Lock-based approach.**

I. Introduction

Spindles are a pattern of brain waves that occur during sleeping periods, with less eye movement. Simply put, sleep spindles are like short bursts of the brain activity. These occur in non-rapid eye movement (NREM) sleep, particularly in stage 2 of the sleep cycle. Such oscillations are recognized by electroencephalogram (EEG) signals in the 12–16 Hz frequency range. The EEG Signals play a key role for the human brain in processes such as memory consolidation, sensory integration, and overall cognitive function. The importance of sleep spindle detection lies in analyzing sleep patterns, diagnosing sleep disorders, and assessing neurological health.

Now, such spindle detection mechanism is affected by factors such as eye movements. The eye movements are recorded through Electrooculogram (EOG) signals. Spindle detection can be difficult attributing to eye movements during transitions between sleep stages. These introduce noise in the EEG data. We isolate spindles from background EEG activity and reduce the impact of such artifacts, meaningful insights can be derived from the brain's activity during sleep.

The identification and classification of sleep spindles within EEG are important to understand our sleep patterns. It is beneficial to understand such influence of EEG spindle detection on neurological health. Now, machine learning models (particularly neural networks) have been widely explored for spindle detection. Some of these methods have extremely high computational demands of training and evaluation. This is a challenge on large-scale EEG datasets and real-time data.

Our project aims to address such challenges by optimizing the training and evaluation processes of a neural network for binary spindle classification. We consider a sequential baseline approach and a parallelized multiprocessing approach. We aim to demonstrate advancements in computational efficiency. It should be done in such a way that model accuracy should not be affected. These enhancements in training and evaluation times can be useful for fast processing of large datasets and facilitate real-time applications like such as sleep monitoring systems and neurofeedback therapies. The project approach has a comprehensive pipeline that integrates data preprocessing, model training, model evaluation, and memory management.

II. Aims and Objectives

The primary objectives of this project are:

- To design and implement an efficient pipeline for preprocessing EEG data and identifying sleep spindles.
- To develop a binary classification neural network for spindle detection.
- To explore and assess parallel processing techniques to achieve parallelism for the neural network training and evaluation process.
- To benchmark and compare the proposed parallel solution against a simple sequential baseline implementation to find betterment in throughput.

III. Related Work

There have been many studies that explore methodologies for detecting sleep spindles in the EEG (electroencephalogram) data. Such techniques include the application of deep learning techniques like wavelet transforms and the integration of EOG (Electrooculogram) signals that help to improve detection accuracy. Some of the approaches that inspired us to do this project use signal processing techniques such as wavelet transforms and short-time Fourier transforms. These are frequently applied for time-frequency analysis, and are useful for time-series data. The integration of the electrooculogram (EOG) signals with the EEG data is a notable method to differentiate spindles from artifacts or noise caused by eye movements. Some of the more advanced frameworks

utilize feedback-controlled stimulation and topographical mapping to analyze spindle detection. These diverse studies aim to improve the accuracy of spindle detection. These are helpful if we consider noise-prone environments, such as EOG signals.

IV. Solution Approach

Our solution for these challenges is designed in many components, each component addressing specific stages of the project. The key modules are as follows:

- **EDA and Preprocessing:** This Jupyter notebook is used to load the two spindle datasets. We combine two datasets for better quality of data. After this, we preprocess the data to use it for training and analysis.
- **training_sequential.py:** Our baseline model uses our combined dataset and processes the EEG/EOG signals with StandardScaler. The training was done through a TimeSeriesAutoencoder with encoder-decoder architecture. It compresses input to 32 dimensions and reconstructs it. The process then uses the Adam optimizer over 100 epochs in batches of 32 which runs sequentially on a single thread.
- **training_parallel.py:** For the parallelization aspect, we split the dataset into equal partitions across multiple CPU cores (up to 8 workers). Each worker processes its work independently using a shared memory model. A progress monitoring system is maintained that tracks the training metrics through a multiprocessing queue. This also ensures synchronization between updates across workers.
- **eval_sequential.py:** Evaluates and compares the sequential approach using graphical analysis and execution results, highlighting the benefits of parallelization.
- **eval_parallel.py:** Evaluates and compares the parallel implementation using performance metrics, focusing on the benefits of parallelization over the sequential approach.
- **run_memory_tests.py:** This module manages the memory usage during training and evaluation phases, specifically on CPU system. We ensure that all operations and multiprocessing tasks are safely terminated and worker threads do not leave behind any memory allocations.
- **training_bench.py:** Benchmarks training times for both sequential and parallel implementations. It evaluates speedup, parallel fraction, and efficiency by varying the number of CPU cores.
- **eval_bench.py:** Benchmarks evaluation throughput for different batch sizes in both sequential and parallel implementations. It highlights the impact of parallelization on inference speed.

A. Overview of the Process

Our parallelization approach primarily focuses on training and evaluating a time-series autoencoder that performs EEG spindle detection in Python. We leverage the PyTorch library and its multiprocessing mechanism to achieve efficient parallelization for both tasks. The dataset has been obtained from Kaggle (https://www.kaggle.com/datasets/jbouv27/eeg/data), consisting of sequential EEG and EOG signals. These signals are preprocessed and divided into smaller sequences suitable for training and evaluation.

For the training part, the neural network model is executed across multiple processes, with each process handling a portion of the dataset. This parallelization achieves efficient utilization of CPU cores and ensures consistent updates to the model parameters using gradient averaging. For the evaluation part, the trained model is assessed by splitting the dataset into partitions and running parallel evaluation processes. Each process independently computes the reconstruction loss for its part subset of the dataset. It is aggregated for overall performance metrics.

B. Key Techniques and Methods

1) Model Architecture:

- For training the neural network, a TimeSeriesAutoencoder class is implemented. It is made up of an encoder to compress the input data and a decoder that reconstructs it. The encoder uses many dense layers to reduce dimensionality and the decoder reconstructs the original input.
- The loss function (MSELoss) is used to measure the reconstruction error. It helps the model to learn meaningful representations of the data in the training phase.

2) Data Handling and Parallelization:

- We have built a custom TimeSeriesDataset class, which preprocesses the dataset by performing normalization of some key signal columns. It converts them into fixed length sequences. Shared memory (share_memory) is used for efficient data access across multiple processes.
- For training the model, the final perprocessed dataset is divided among multiple worker processes using PyTorch's torch.multiprocessing and DataLoader. Each worker process independently performs training on its portion (subset) of data.
- For evaluation of the model, a similar approach as above is used. The dataset is again partitioned, with each process evaluating a distinct subset. We ave implemented a Queue mechanism, that ensures efficient communication of results of worker processes back to the main process.

3) Synchronization and Gradient Averaging (Training):

- During the training phase, each of the workers calculates gradients for its data subset. To ensure consistency, gradients are averaged across all processes using a custom average_gradients function.
- A multiprocessing Lock structure is used to manage updates to the shared loss array. This approach ensures that workers do not overwrite each other's results.

4) Results Aggregation (Evaluation):

- For evaluation, each process computes the total loss and batch count for its assigned dataset partition. The computed results are communicated to the main process using a multiprocessing Queue.

C. Execution

1) Training:

- **Dataset Partitioning:** The final dataset obtained after preprocessing is divided among workers, with each worker independently loading and processing its subset using multi-threaded "DataLoader" instances.
- **Gradient Computation:** The worker processes compute gradients in parallel for their subsets of the data. These gradients are then synchronized across workers using a lock-free averaging mechanism.
- **Loss Tracking:** The loss tracking is done using a shared loss array, which tracks the training progress across all the epochs. We have implemented fine-grained locking, that helps to synchronize updates to the shared-loss array without significant contention on the array.
- **Progress Monitoring:** To communicate the results and monitor the progress of the individual workers, a separate process monitors training progress. A lock-free queue-based communication mechanism to is implemented that prints epoch-wise updates to the console.

2) Evaluation:

- **Dataset Partitioning:** As done in training, the dataset is similarly split into partitions, each partition is assigned to an independent worker process for evaluation. Each process uses a "DataLoader" instance to load data in batches.
- **Model Evaluation:** Now, for the evaluation each worker process loads the trained model and performs "forward passes" on its dataset partition and computes the reconstruction loss.
- **Results Aggregation:** Each process sends its computed loss and batch counts to the main process via a queue. The main process aggregates these results and calculates the average reconstruction loss.
- **Process Management:** Processes are efficiently managed using `torch.multiprocessing`, with the number of processes dynamically determined based on available CPU cores.

D. Comparison of Training and Evaluation Parallelization

As described in above sections, the parallelization techniques for training and evaluation share some similarity, however they differ in synchronization requirements and workload distribution. A summary table for both training and evaluation parallelization methods is provided below:

E. Execution and Results

- The parallel training is initiated by the "train_parallel" function, which creates multiple worker processes and a progress monitor process. Each worker trains the model on a portion of the dataset. The training progress is displayed in real time on the console.
- The trained model and the associated parameters are saved for future use in "timeseries_autoencoder_par.pth".
- The approach is optimized for macOS systems by using the spawn method to initialize subprocesses.

TABLE I
SUMMARY OF PARALLELIZATION: TRAINING

Component	**Parallelization Method**	**Synchronization Method**
Data Loading	Partitioned among workers, multi-threaded loaders	Independent for each worker
Gradient Computation	Parallel across workers	Lock-free, synchronized via averaging
Loss Tracking	Distributed computation	Fine-grained locking
Progress Monitoring	Separate process with queue-based communication	Lock-free via queue

TABLE II
SUMMARY OF PARALLELIZATION: EVALUATION

Component	**Parallelization Method**	**Synchronization Method**
Dataset Partitioning	Split among processes	Independent for each process
Model Evaluation	Independent computation on each process	None (read-only evaluation)
Loss Calculation	Distributed across processes	Aggregated via queue
Results Aggregation	Gathered via queue	Lock-free with queue
Process Management	Multiple processes using multiprocessing	Managed by multiprocessing library

Our solution to make training and evaluation parallel combines the strengths of neural networks and parallel computing. It demonstrates an important application of multiprocessing for large-scale time series analysis for EEG spindle detection. Methods and techniques, such as shared memory, lock-based synchronization, and gradient averaging, ensure both computational efficiency and accuracy in training.

V. EXPERIMENTAL RESULTS

A. Experimental Setup

The experiments were carried out on an 8-core macOS processor for sequential and parallel implementations. For parallel training, we have used Python's multiprocessing capabilities with 4, 8, and 12 threads for training and evaluation benchmarks.

B. Performance Metrics

Performance was evaluated using the following metrics:

- **Training Time**: The time taken to complete the training process for both sequential and parallel implementations of the model.
- **Speedup**: Speedup is the ratio of sequential training time to parallel training time.
- **Throughput**: The number of samples processed per second for both sequential and parallel implementations during evaluation.

C. Results and Analysis

a) Training Time and Speedup Analysis: The training time for sequential implementation was 2551.20 seconds.

Parallel implementations achieved significant reductions in training time, with observed speedups of 4.54x, 4.21x, and 4.15x using 4, 8, and 12 threads, respectively. Figure 1 illustrates the training times and speedup achieved.

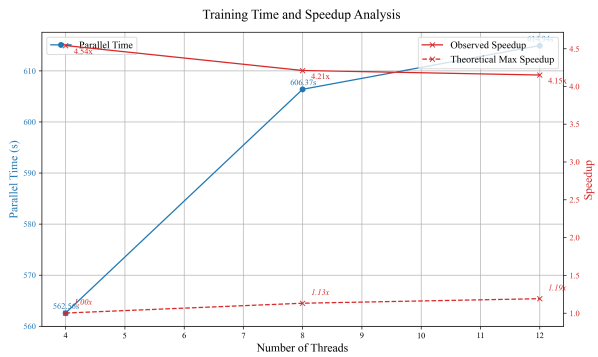

Fig. 1. Training Time and Speedup Analysis

b) Sequential vs Parallel Training: Parallel training significantly outperformed sequential training across all tested configurations. With 4, 8, and 12 threads, the parallel training times were 562.56s, 606.37s, and 614.94s, respectively, compared to the sequential time of 2551.20s. Figure 2 highlights the comparison.

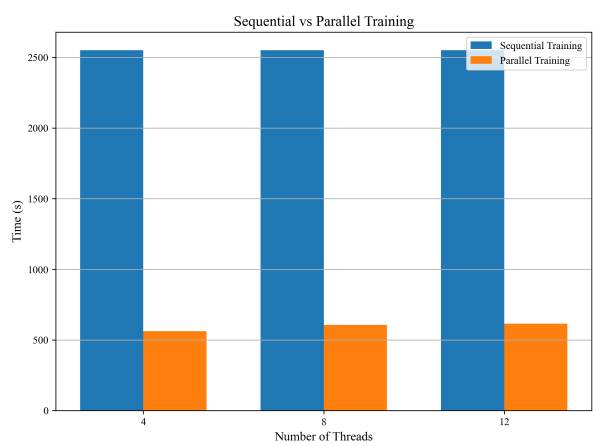

Fig. 2. Sequential vs Parallel Training

c) Sequential vs Parallel Evaluation: Evaluation throughput also demonstrated notable improvements with parallel execution. Sequential evaluation was considerably slower compared to parallel execution with 4, 8, and 12 threads. Figure 3 shows the evaluation time comparisons.

d) Batch Size and Throughput: The throughput analysis for different batch sizes (32, 64, 128, 256) revealed that smaller batch sizes yielded higher speedups. For instance, batch size 32 achieved a speedup of 6.19x with 7 threads and 8.81x with 16 threads. Figure 4 depicts the throughput, while Figure 5 illustrates speedups for varying batch sizes.

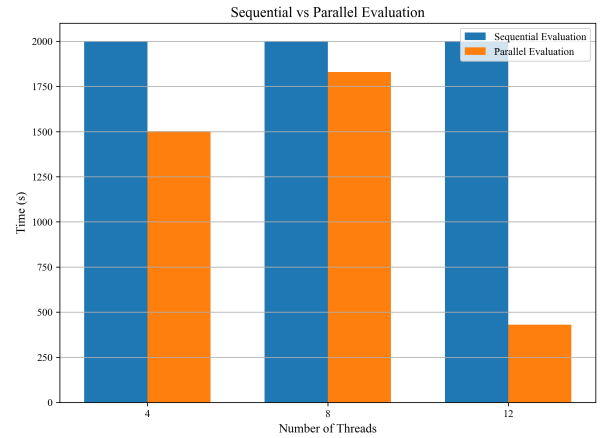

Fig. 3. Sequential vs Parallel Evaluation

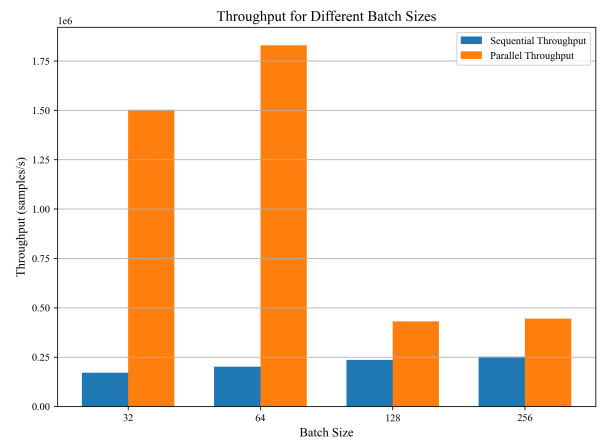

Fig. 4. Throughput for Different Batch Sizes

D. Summary of Executions

We infer that the parallel implementations have helped to reduce the training and evaluation times while maintaining model accuracy. The approach of using smaller batch sizes exhibited higher throughput gains. This helps us understand the importance of optimizing batch configurations for parallel training. From these results, it gives a clear idea of the potential of parallel computing in EEG signal processing and spindle detection tasks.

VI. Conclusion and Future Work

Our multiprocessing programming project successfully demonstrated the strength of parallel processing techniques to reduce training and evaluation times for EEG spindle detection. By using Python's multiprocessing library and techniques, we achieved up to 4.54x speedup in training time and 8.81x speedup in evaluation throughput compared to the sequential approach. The results obtained from benchmarking the sequential and parallel approaches confirm the feasibility of parallelizing time series neural network tasks while maintaining model accuracy. We can see the effectiveness of

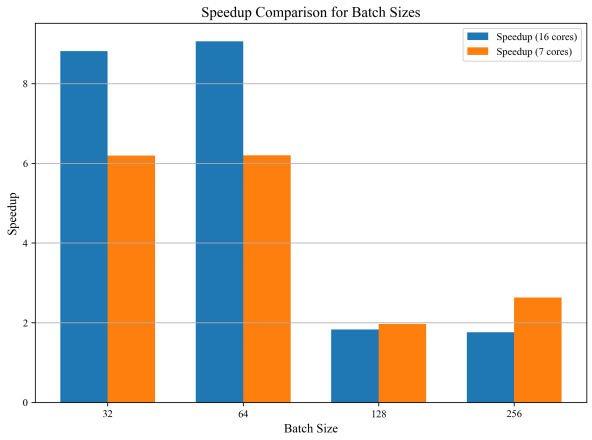

Fig. 5. Speedup Comparison for Batch Sizes

combining parallel computing with neural network-based EEG signal processing for real-world applications.

Our future efforts to improve this approach for faster training and evaluation of EEG spindle detection will focus on exploring lock-free synchronization mechanisms. This will help to further improve speedup and reduce overhead in parallel implementations. Along with this, we also aim to optimize the parallelization strategy for larger datasets and use more complex neural network architectures like a transformer model. We will try to explore GPU-based parallelism and advanced memory management techniques, such as Safe Memory Reclamation (SMR). This could further boost performance, which is expected to give around 8x speedup for training of the neural network.

Acknowledgments

We would like to thank Professor Binoy Ravindran for his support and guidance throughout the project development. We are inspired by his teachings and will try to implement some concepts we learned from his course.

References

1) Wei, L., Ventura, S., Mathieson, S., Boylan, G.B., Lowery, M. and Mooney, C., 2021. Spindle-AI: sleep spindle number and duration estimation in infant EEG. *IEEE Transactions on Biomedical Engineering*, 69(1), pp.465-474.
2) Kaulen, L., Schwabedal, J. T. C., Schneider, J., Ritter, P., and Bialonski, S., 2022. Advanced sleep spindle identification with neural networks. *Scientific Reports*, 12, Article 7686.
3) Dursun, M., Özşen, S., Yücelbaş, C., Yücelbaş, Ş., Tezel, G., Küçüktürk, S., and Yosunkaya, Ş., 2017. A new approach to eliminating EOG artifacts from the sleep EEG signals for the automatic sleep stage classification. *Neural Computing and Applications*, 28, pp.3095-3112.
4) Kayabekir, M., and Yağanoğlu, M., 2024. SPINDILOMETER: a model describing sleep spindles on EEG signals for polysomnography. *Physical and Engineering Sciences in Medicine*, pp.1-13.
5) Patti, C.R., Chaparro-Vargas, R., and Cvetkovic, D., 2014, August. Automated sleep spindle detection using novel EEG features and mixture models. In *2014 36th Annual International Conference of the IEEE Engineering in Medicine and Biology Society* (pp. 2221-2224). IEEE.
6) Al-Salman, W., Li, Y., and Wen, P., 2019. Detecting sleep spindles in EEGs using wavelet Fourier analysis and statistical features. *Biomedical Signal Processing and Control*, 48, pp.80-92.
7) Schimicek, P., Zeitlhofer, J., Anderer, P., and Saletu, B., 1994. Automatic sleep-spindle detection procedure: aspects of reliability and validity. *Clinical Electroencephalography*, 25(1), pp.26-29.
8) Heileman, G.L., Georgiopoulos, M., and Roome, W.D., 1992. A general framework for concurrent simulation of neural network models. *IEEE Transactions on Software Engineering*, 18(7), pp.551-562.
9) Gunther, S., Ruthotto, L., Schroder, J.B., Cyr, E.C., and Gauger, N.R., 2020. Layer-parallel training of deep residual neural networks. *SIAM Journal on Mathematics of Data Science*, 2(1), pp.1-23.
10) Tagluk, M.E., Sezgin, N., and Akin, M., 2010. Estimation of sleep stages by an artificial neural network employing EEG, EMG, and EOG. *Journal of Medical Systems*, 34, pp.717-725.